

Reviewer Pack — Operator-SoS 4-Trace Gap Lower-Bounds Blocks-Order Read-Twic...

Entry 0ecd5f06ea81 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 02:59:10 UTC

Operator-SoS 4-Trace Gap Lower-Bounds Blocks-Order Read-Twice BP for IP₂

Entry ID: 0ecd5f06ea81 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 02:59:10 UTC

1. Conjecture

Field A (mathematical branch): Sum-of-Squares / Lasserre hierarchy on operator-valued layer-product tensor moments (Putinar-Helton-style non-commutative SoS pseudo-trace invariants applied to layer-difference matrices of branching programs — a corner combining moment-SoS literature with BP structural invariants, with <5 papers on the BP side) **Field B** (complexity object): Read-twice oblivious branching program size for IP₂ (the open BP_READTWICE separation of Borodin-Razborov-Smolensky 1993)

Statement:

Let P be a read-twice oblivious BP on $2n$ Boolean variables $(x_1, \dots, x_n, y_1, \dots, y_n)$ with width w and length $4n$, layer transition matrices $M_j(0), M_j(1) \in \{0,1\}^{w \times w}$, and layer-difference matrices $N_j := M_j(1) - M_j(0)$; for each variable v with its two read-layers (a_v, b_v) form the paired operator $P_v := N_{\{a_v\}} N_{\{b_v\}} \in R^{w \times w}$, and define the operator-SoS 4-trace gap $\rho(P) := \log_2 \max_{u \neq v} |\langle P_u, P_v \rangle_F| / w^2$, where $\langle A, B \rangle_F = \text{tr}(A B^T)$. CONJECTURE: (Upper anchor, trivial via SoS-Cauchy-Schwarz) $\rho(P) \leq 2 \cdot \log_2 \text{width}(P) + 2$ for every read-twice oblivious BP P ; (Lower bound, non-trivial) for every BLOCKS-ORDERED read-twice oblivious BP P (all x -layers preceding all y -layers in each of the two reads) that computes $\text{IP}_2(x, y) = \bigoplus_i x_i y_i$ with bias $\geq 1/2 + n^{-2}$, $\rho(P) \geq n/16 - 4 \cdot \log_2 n$. Equivalently, any such blocks-ordered read-twice BP for IP₂ has width $\geq 2^{\{n/40\}}$ for all sufficiently large n ; a single small-width blocks-ordered read-twice BP for IP₂ with $\rho(P) < n/16 - 4 \cdot \log_2 n$ falsifies the conjecture.

Rationale (proposer's reasoning):

The paired operator $P_v = N_{\{a_v\}} N_{\{b_v\}}$ packages the two reads of variable v into a single non-commutative SoS-style moment, and the Frobenius pairing $\langle P_u, P_v \rangle_F$ is exactly the kind of trace identity that survives Lasserre relaxation at level 2 yet remains computable directly from the BP. For IP₂ the algebraic obstruction that forces exponential read-once size — namely that x and y blocks cannot be summarized in fewer than 2^n states without bilinear cancellation — should reappear in the read-twice setting as a forced spread of mass across many variable-pairs in the Gram matrix of $\{P_v\}$, producing a heavy off-diagonal entry. Because ρ depends on the specific BP layer matrices (not the truth table) it cannot collapse to a function-symmetric invariant, side-stepping the natural-proofs trap that killed the prior Lasserre-rank attempt in this approach.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 238621f6d8b52a30

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all (n, w) cells ($n \in \{6, 10, 16, 24, 32, 40\}$, $w \in \{2, 4, 8\}$; 30 random read-twice BPs each = 540 random instances) the trivial upper anchor $\rho(P) \leq 2 \cdot \log_2 w + 2$ must hold; additionally for each $n \in \{4, 6, 8, 10\}$ the explicit blocks-ordered IP₂ BP of width 2^n must satisfy $\rho(P) \geq n/16 - 4 \cdot \log_2 n$. Support requires 100% of random instances obeying the anchor and $\geq 3/4$ IP₂ instances obeying the lower bound.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.94	SAFE	SAFE
NATURAL PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - read-twice branching program lower bound inner product IP2 - non-commutative sum-of-squares branching program width lower bound-oblivious branching program layer transition matrix Frobenius trace inner product

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import json

def matrix_multiply(A, B):
    w = len(A)
    result = [[0 for _ in range(w)] for _ in range(w)]
    for i in range(w):
        for j in range(w):
            for k in range(w):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_subtract(A, B):
    w = len(A)
```

```

    return [[A[i][j] - B[i][j] for j in range(w)] for i in range(w)]

def matrix_transpose(A):
    w = len(A)
    return [[A[j][i] for j in range(w)] for i in range(w)]

def frobenius_inner_product(A, B):
    w = len(A)
    product = matrix_multiply(A, matrix_transpose(B))
    trace = sum(product[i][i] for i in range(w))
    return trace

def generate_bp(w, n, seed):
    random.seed(seed)
    M = [random.choice([0, 1]) for _ in range(2*n)]
    N = [matrix_subtract(M[j], M[j-1]) if j > 0 else M[j] for j in range(2*n)]
    return N

def compute_rho(P, w):
    n = len(P) // 2
    max_inner = 0
    for u in range(n):
        for v in range(u+1, n):
            inner = abs(frobenius_inner_product(P[u], P[v]))
            if inner > max_inner:
                max_inner = inner
    if max_inner == 0:
        return 0.0
    return math.log2(max_inner) - 2 * math.log2(w)

def run_trial(seed):
    n_values = [6, 10, 16, 24, 32, 40]
    w_values = [2, 4, 8]
    metric_values = []
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        for w in w_values:
            N = generate_bp(w, n, seed)
            P = [matrix_multiply(N[2*i], N[2*i+1]) for i in range(n)]
            rho = compute_rho(P, w)

            if rho > 2 * math.log2(w) + 2:
                conjecture_holds = False
                counterexample = f"Upper bound violated for n={n}, w={w}, rho={rho}"
                break

            metric_values.append(rho)
            instances_tested += 1

        if not conjecture_holds:
            break

    if conjecture_holds:

```

```

for n in [4, 6, 8, 10]:
    w = 2 ** n
    N = generate_bp(w, n, seed)
    P = [matrix_multiply(N[2*i], N[2*i+1]) for i in range(n)]
    rho = compute_rho(P, w)

    lower_bound = n / 16 - 4 * math.log2(n)
    if rho < lower_bound:
        conjecture_holds = False
        counterexample = f"Lower bound violated for n={n}, w={w}, rho={rho}"
        break

    metric_values.append(rho)
    instances_tested += 1

return {
    "metric_name": "operator-SoS 4-trace gap",
    "metric_value": sum(metric_values) / len(metric_values) if metric_values else 0.0,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179, 181, 187, 191, 193, 197, 199, 211, 223, 227, 229, 233, 239, 241, 251, 257, 263, 269, 271, 277, 281, 283, 293, 307, 311, 313, 317, 331, 337, 347, 349, 353, 359, 367, 373, 379, 383, 389, 397, 401, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523, 527, 539, 541, 547, 557, 563, 569, 571, 577, 587, 593, 599, 601, 607, 613, 617, 619, 623, 629, 631, 637, 641, 643, 647, 653, 659, 661, 667, 671, 673, 677, 683, 687, 691, 697, 701, 703, 707, 713, 719, 727, 733, 737, 743, 751, 757, 761, 769, 773, 779, 787, 797, 803, 809, 811, 817, 821, 823, 827, 829, 833, 837, 851, 853, 857, 859, 863, 869, 877, 881, 883, 887, 893, 897, 901, 907, 911, 913, 917, 919, 923, 929, 931, 937, 941, 943, 947, 953, 959, 961, 967, 971, 973, 977, 983, 989, 991, 993, 997, 1000]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        result["seed"] = seed
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results if "metric_value" in r]
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results) if results else 0.0

    if all(r["conjecture_holds"] for r in results):
        mean = sum(metric_values) / len(metric_values) if metric_values else 0.0
        std = (sum((x - mean) ** 2 for x in metric_values) / len(metric_values)) ** 0.5 if metric_values else 0.0
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8f1e5764.py", line 113, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8f1e5764.py", line 69, in run_trial
    N = generate_bp(w, n, seed)
        ^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8f1e5764.py", line 44, in generate_bp
    N = [matrix_subtract(M[j], M[j-1]) if j > 0 else M[j] for j in range(2*n)]
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8f1e5764.py", line 28, in matrix_subtra
    w = len(A)
        ^^^^^

```

TypeError: object of type 'int' has no len()

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a TypeError in matrix_subtract before producing any data, so neither the upper-anchor nor the IP_2 lower-bound criterion was evaluated. No RESULT line was emitted and the pre-registered support condition cannot be assessed. | next: Fix the matrix representation bug in generate_bp/matrix_subtract (ensure M[j] is a 2D list/array of shape w×w, not an int) and re-run the 540 random instances plus the 4 explicit IP_2 BPs.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	239299
2	preregistration	claude_max	opus	0	0	7184
3	novelty	claude_max	opus	0	0	3757
4	novelty	claude_max	opus	0	0	6952
5	test_gen	mistral	codestral-latest	0	0	312197
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	224007
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	273134
8	test_gen	mistral	codestral-latest	0	0	312263
9	judge	claude_max	opus	0	0	4829

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1383621 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 2, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/0ecd5f06ea81.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0ecd5f06ea81.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0ecd5f06ea81.tar.gz` (if generated)